



UNIVERSITÄT
LEIPZIG

Software Engineering 2025/26 – Tutorium

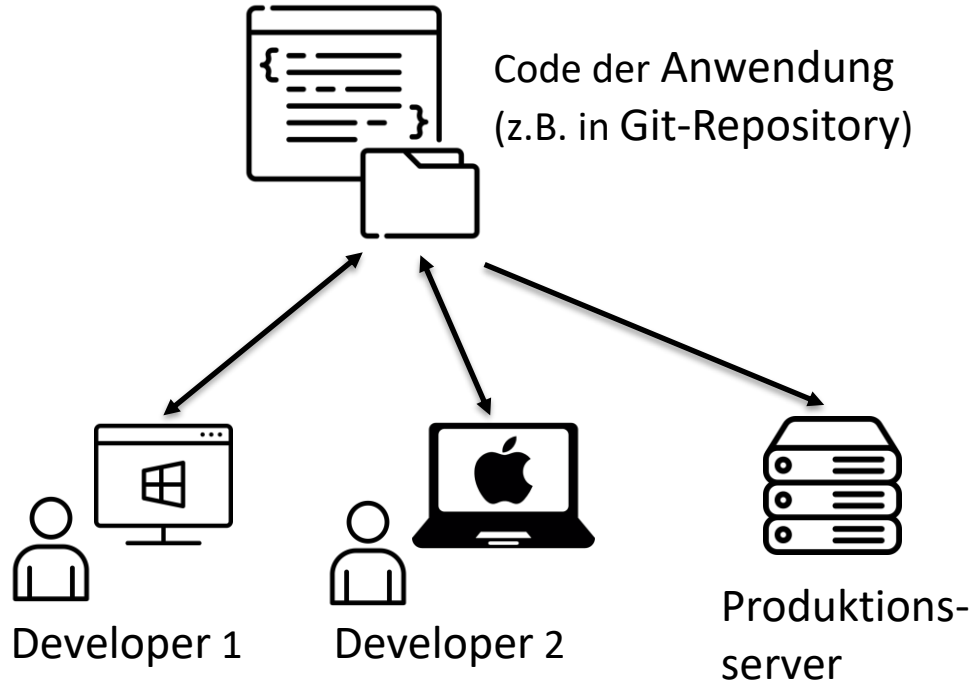
Prof. Dr.-Ing. Norbert Siegmund
Tristan Scholz & Tobias Matzke



Tutorium 3 – Containerisierung mit Docker

- 1. Motivation
 - Wozu braucht man Docker?
 - Welche Problematik löst es?
- 2. Grundlagen von Docker
 - Was ist ein Container?
 - Grundkonzepte von Docker
 - QUIZ
- 3. Erste Schritte (Praxisteil 1)
- 4. Weiterführendes Wissen (Praxisteil 2)
 - Docker Volumes
 - Docker Compose
 - Best Practices

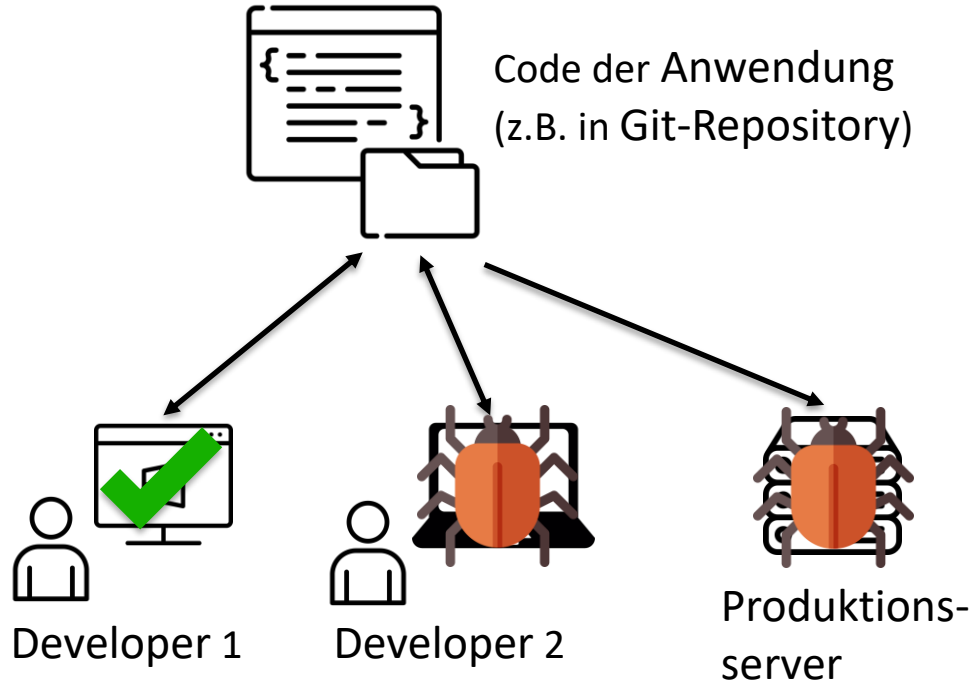
1. Motivation



Ausgangssituation:

- Mehrere Computer haben Zugriff auf den gemeinsamen Quellcode
- Jeder Computer wandelt **individuell** den Code in ausführbare Anwendung um
- Computer haben **unterschiedliche** Hardware/Betriebssysteme/...

1. Motivation

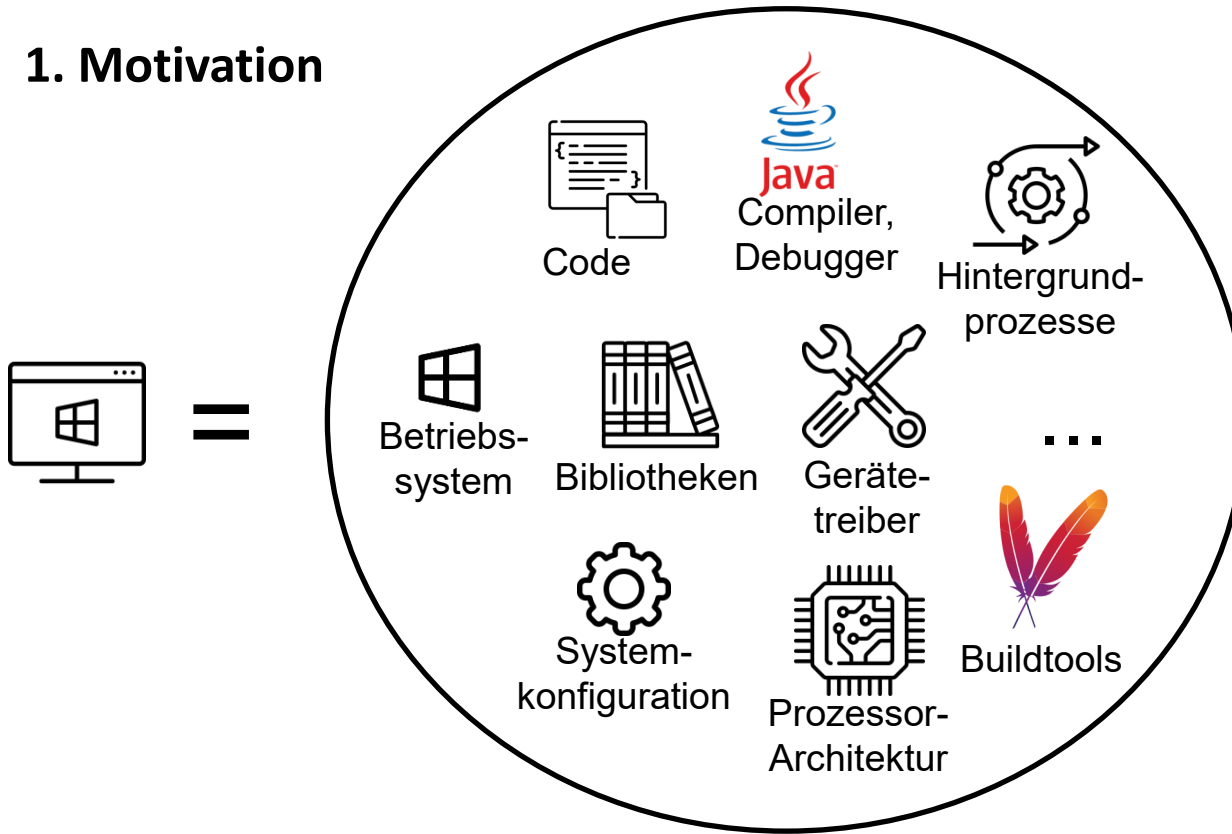


„Works-on-my-Machine-Problemik“

- Anwendung zeigt je nach Hostsystem **unterschiedliches Verhalten**
- Auf einem Computer läuft alles reibungslos, auf einem anderen treten Bugs auf
- **Extrem problematisch**, Anwendung soll sich im besten Fall immer gleich verhalten

→ **Woran liegt das?**

1. Motivation



Grund: unterschiedliche...

- Systemarchitekturen
- Konfigurationen
- Bibliotheken
- ...

Beispiel 1: Unterschiedliche Betriebssysteme

Rechner des Developers: **Windows**

```
File file = new File("\\data\\customers.csv"); //Pfad zur Datei
```

→ Laden der Daten funktioniert problemlos



Server in der Produktionsumgebung: **Linux**

→ **Fehler**, Datei mit Kundendaten kann nicht gefunden werden



Grund: Pfade unter Windows werden mit \ getrennt, unter Linux mit /

Beispiel 2: Unterschiedliche Betriebssysteme

Rechner des Developers: **Windows**

```
File file = new File(pathname:"Customers.csv"); //Pfad zur Datei
```

→ Laden der Daten funktioniert problemlos



Server in der Produktionsumgebung: **Linux**

→ **Fehler**, Datei mit Kundendaten kann nicht gefunden werden



Grund: Name der Datei ist eigentlich **customers.csv**, Linux-Dateisystem ist case-sensitive, Windows nicht

Beispiel 3: Versionsunterschiede

Rechner des Developers: **Java Version 17**



```
String typeOfDay = switch (day) {  
    case 1, 2, 3, 4, 5 -> "Weekday";  
    case 6, 7 -> "Weekend";  
    default -> "Invalid day";  
};
```

Alter Produktionsserver: **Java Version 9**



→ Build schlägt fehl

Grund: Notation mit -> bei case-Verzweigungen wurde erst in **Java 14** eingeführt

(alte Notation funktionierte mit : und **break;**)

Beispiel 4: unterschiedliche Systemeinstellungen (Sprache)

Aufgabe: Fehlermeldungen einer Postgres-Datenbank in Kategorien einteilen

```
[SUCCESS] Successfully classified 5429 Errors from table ERRORDATA.  
[INFO] Fehlerklassenzusammenfassung:  
Fehlerklasse | Vorkommen  
-----|-----  
Doppelter_Schlüsselwert | 2173  
Doppelter_Attributwert | 2134  
Fremdschlüsselbedingung_verletzt | 1010  
Unerlaubter_NULL_Wert | 100  
Attributwert_nicht_sinnvoll | 12  
nicht_zugeordnet | 1
```

Bsp. Für Kategorisierung: `if (fehlermeldung.contains("duplicate key")) { return "Doppelter_Schlüsselwert"; }`

Developer 1: Betriebssystem auf **Englisch** gestellt

Fehlermeldung: **duplicate key** value violates unique constraint »kunde_pkey«



Developer 2: Betriebssystem auf **Deutsch** gestellt

Fehlermeldung: **doppelter schlüsselwert** verletzt unique-constraint »kunde_pkey«



Lösungsansatz

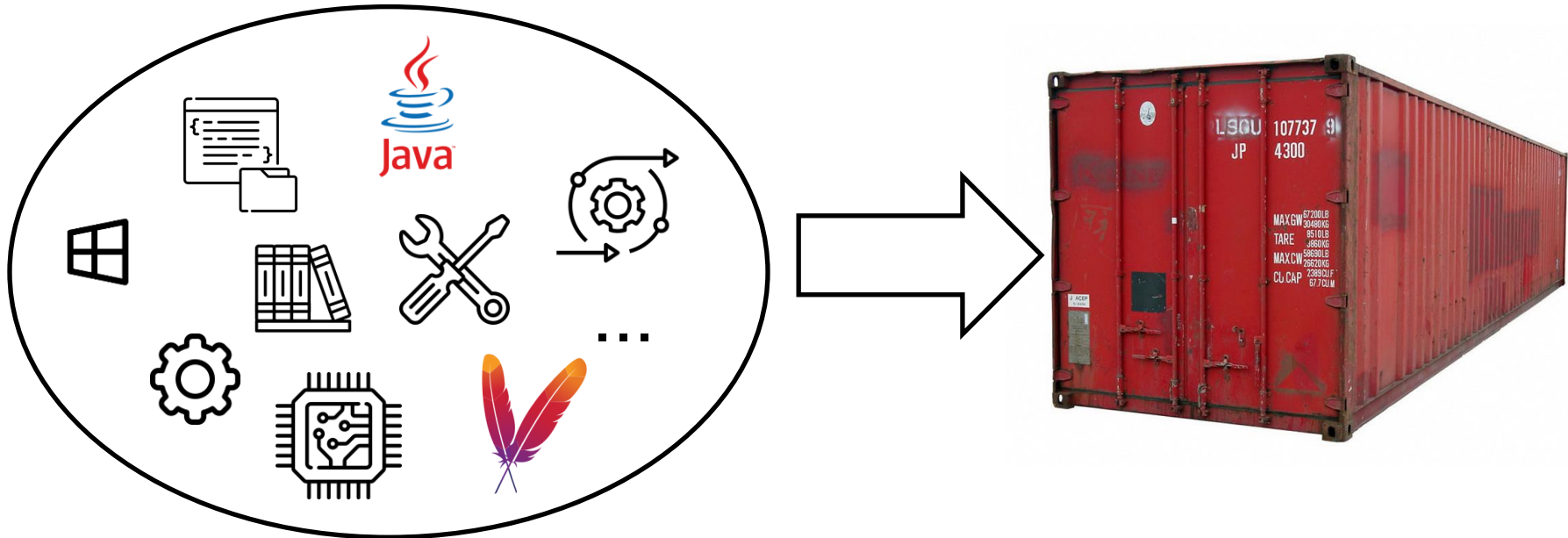
Gesucht:

- Technologie, die dafür sorgt, dass sich Anwendung auf jedem Rechner gleich verhält
- Soll unabhängig von Hardwareplattform, Betriebssystem, ... funktionieren

Idee: Alle für die Anwendung relevanten Komponenten werden **STANDARDISIERT** und in einem Container **GEBÜNDELT** bereitgestellt

2. Grundlagen von Docker – Was ist ein Container?

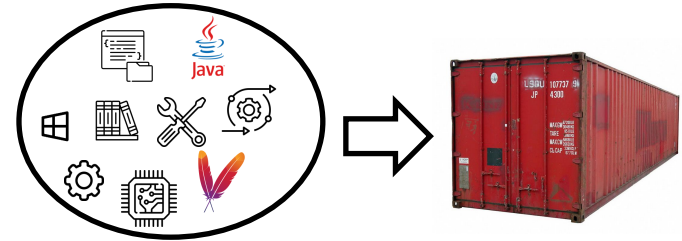
→ Idee: Alle für die Anwendung **relevanten Komponenten** werden **STANDARDISIERT** und in einem „Container“ **GEBÜNDELT** bereitgestellt



2. Grundlagen von Docker – Was ist ein Container?

Alle für die Anwendung **relevanten Komponenten** werden **STANDARDISIERT** und in einem „Container“ **GEBÜNDELT** bereitgestellt

- Merkmale eines Containers
 - „Eigenes, in sich geschlossenes Ökosystem“
 - Vom Betriebssystem isolierte Laufzeitumgebung
 - Enthält alle nötigen Programme & Bibliotheken
 - Versionen sind jeweils klar definiert
 - Portabel: kann auf jedem beliebigen Rechner laufen

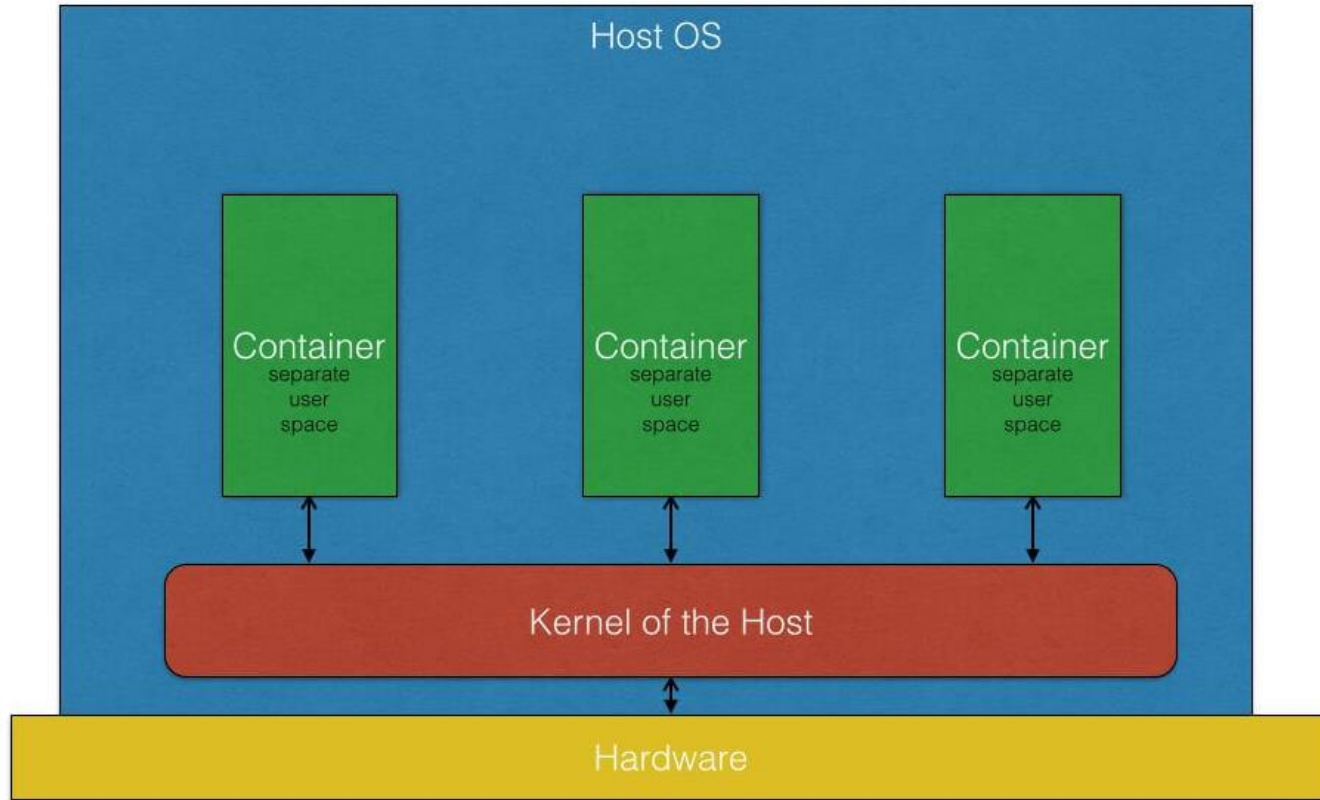


→ **Problem gelöst:** auf jedem Rechner läuft exakt der gleiche Container, unabhängig von Betriebssystem, Konfiguration, ...

2. Grundlagen von Docker

- Was benötigt man, um eine containerisierte Anwendung auf einem Rechner auszuführen?
 - Den **Container**, in welchem sich die Anwendung befindet (mehr dazu später)
 - Eine „**Container-Engine**“: Software, welche Container erstellt, ausführt, verwaltet
 - (Engine verwaltet **Ressourcen** der Container und stellt **Isolation** sicher)
- In diesem Tutorium nutzen wir die Docker-Engine

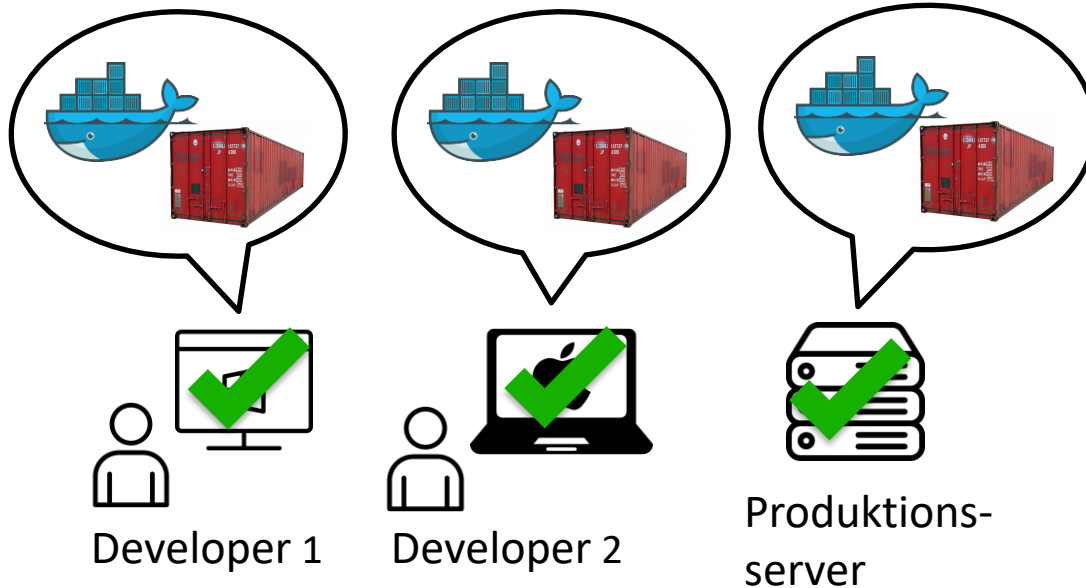




Operating System/Container Virtualization

Grafik: <https://blog.risingstack.com/operating-system-containers-vs-application-containers/> (Zugriff am 22.10.2025 10:16)

2. Grundlagen von Docker



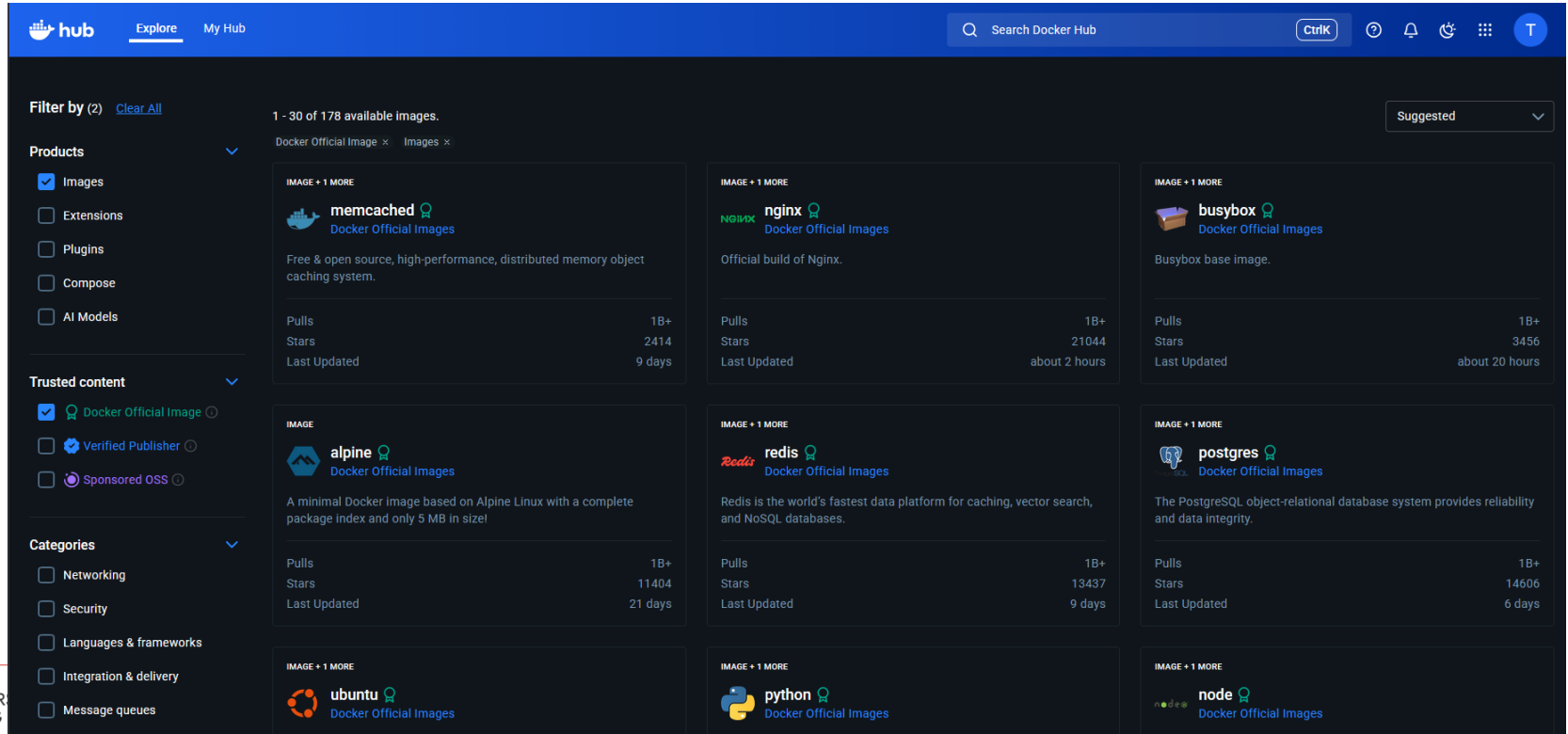
- Auf jedem Rechner befindet sich die Docker Engine und die containerisierte Anwendung
- **Ziel erreicht:** Anwendung verhält sich nun überall gleich
- **Frage:** wie kommen die Container auf den Rechner?

2. Grundlagen von Docker - Begriffe

- ① *Dockerfile*: Textdatei mit **Anweisungen** nach denen das Dockerimage erstellt wird („Bauanleitung“)
- ② ↓ build
- *Dockerimage*: enthält **Momentaufnahme** einer **Anwendung und ihrer Umgebung** (unveränderbare Datei)
- ③ ↓ run
- *Dockercontainer*: laufende **Instanz** eines Dockerimages, führt **Anwendung und Umgebung isoliert** aus
 - *Dockerengine*: Laufzeitumgebung, in welcher der Container läuft

2. Grundlagen von Docker - Begriffe

- **Dockerhub:** Registry (**Bibliothek**) für Dockerimages, enthält unzählige **fertige Dockerimages**, ermöglicht Teilen und Verwalten **eigener** Images



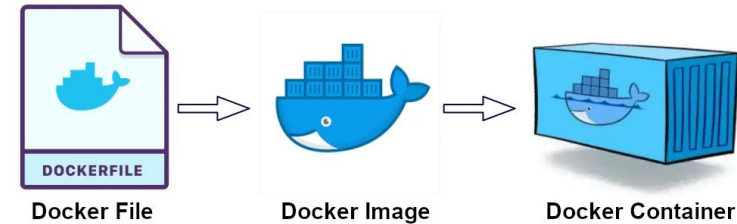
The screenshot displays the Docker Hub web interface. At the top, there's a navigation bar with 'hub', 'Explore', and 'My Hub' tabs, along with a search bar and user profile icons. The main content area is titled 'Filter by (2) Clear All' and shows '1 - 30 of 178 available images'. The left sidebar contains filters for 'Products' (Images, Extensions, Plugins, Compose, AI Models) and 'Trusted content' (Docker Official Image, Verified Publisher, Sponsored OSS). The 'Categories' section on the left lists Networking, Security, Languages & frameworks, Integration & delivery, and Message queues. The main grid displays several Docker images, each with its logo, name, description, and statistics (Pulls, Stars, Last Updated).

Image Name	Description	Pulls	Stars	Last Updated
memcached	Free & open source, high-performance, distributed memory object caching system.	1B+	2414	9 days
nginx	Official build of Nginx.	1B+	21044	about 2 hours
busybox	Busybox base image.	1B+	3456	about 20 hours
alpine	A minimal Docker image based on Alpine Linux with a complete package index and only 5 MB in size!	1B+	11404	21 days
redis	Redis is the world's fastest data platform for caching, vector search, and NoSQL databases.	1B+	13437	9 days
postgres	The PostgreSQL object-relational database system provides reliability and data integrity.	1B+	14606	6 days
ubuntu				
python				
node				

2. Grundlagen von Docker

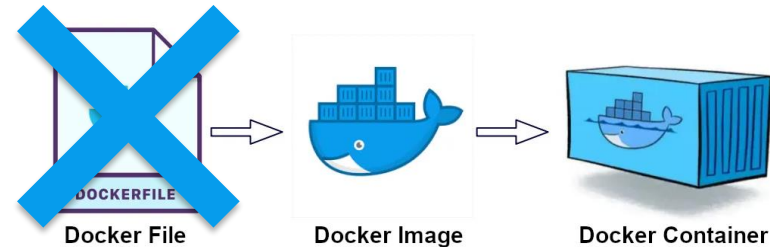
- Variante A: Eigenes Dockerfile

- Dockerimage wird selbst erstellt (per Dockerfile)
- Inhalt: eigene Anwendung (Webserver, Backend-Service, ...)
- Volle Kontrolle über Abhängigkeiten und Bau des Images
- Verwendung: für eigene Softwareprojekte



- Variante B: Fertiges Image verwenden

- Kein eigenes Dockerfile nötig
- Nutzung bereits bestehender Images (→ Docker Hub)
- Meist für Standarddienste genutzt (z.B. Datenbanken)
- Einfach, aber weniger Flexibilität (unproblematisch, da diese bei Standarddiensten meist nicht nötig ist)



Grafik: <https://medium.com/@kuldeepkumawat195/how-to-publish-docker-images-to-docker-hub-f1a40360e0dd> (Zugriff am 23.10.2025 12:40)

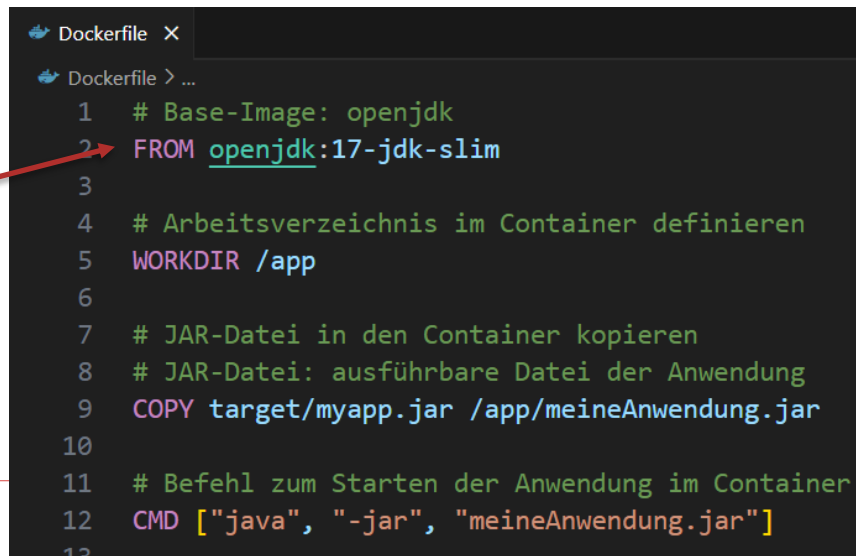
Anmerkungen zu Variante A: Das Base Image

- Grundlage eines eigenen Images ist oft ein **Base-Image**, welches im Dockerfile definiert wird
 - Base Image = Ausgangspunkt, vorgefertigte Systemumgebung („Fundament“)
- vereinfacht Definition des Images erheblich, denn man muss die **Umgebung nicht selbst aufbauen**, sondern **integriert** die **eigene Anwendung** in eine bereits **funktionierende Umgebung**

- mögliche Base-Images:

- openjdk
- node
- ubuntu
- nginx
- postgres
-

Definition des
Base-Images
(inkl.Version)

A screenshot of a code editor showing a Dockerfile. The file has a tab labeled 'Dockerfile' and a breadcrumb 'Dockerfile > ...'. The code is as follows:

```
1 # Base-Image: openjdk
2 FROM openjdk:17-jdk-slim
3
4 # Arbeitsverzeichnis im Container definieren
5 WORKDIR /app
6
7 # JAR-Datei in den Container kopieren
8 # JAR-Datei: ausführbare Datei der Anwendung
9 COPY target/myapp.jar /app/meineAnwendung.jar
10
11 # Befehl zum Starten der Anwendung im Container
12 CMD ["java", "-jar", "meineAnwendung.jar"]
13
```

A red arrow points from the text 'Definition des Base-Images (inkl.Version)' to the 'FROM' instruction on line 2.

Was ist ein Docker-Container?

A) Ein vollständiges Betriebssystem mit eigenem Kernel

B) Eine isolierte Laufzeitumgebung, die alle Abhängigkeiten der Anwendung enthält

C) Ein Cloud-Service zum Speichern von Images

D) Ein Compiler für Java-Anwendungen

Was versteht man unter Container-Isolation?

A) Container teilen Ressourcen uneingeschränkt

B) Container brauchen kein Betriebssystem

C) Container laufen getrennt voneinander und vom Host

D) Container können nicht gelöscht werden

Was ist der Unterschied zwischen einem Image und einem Container?

A) Image enthält nur Metadaten,
Container enthält zusätzlich den
laufenden Prozess

B) Es gibt keinen Unterschied

C) Image = laufende Instanz,
Container = Vorlage

D) Image = Vorlage, Container =
laufende Instanz

Welche der folgenden Aussagen trifft auf ein Dockerfile zu?

A) Es beschreibt, wie ein Docker-Image gebaut wird

B) Es ist ein fertiger Container

C) Es kann nur für Java-Anwendungen verwendet werden

D) Es startet automatisch Docker auf dem Host

Was passiert, wenn man kein Base Image im Dockerfile angibt?

A) Das Image wird leer gebaut, man muss alles selbst installieren

B) Docker wählt automatisch ein Base Image

C) Container kann nicht gestartet werden

D) Das Image ist größer als üblich

Was ist ein wesentlicher Unterschied zwischen Containern und virtuellen Maschinen?

A) Container haben immer ein eigenes Betriebssystem

B) Container teilen sich den Kernel des Hostsystems

C) Container sind schwerer und langsamer als VMs

D) Container können keine Anwendungen ausführen

3. Erste Schritte (Praxisteil 1)

- Ziele:
 - Erster Container (hello-world)
 - Einfacher Webserver-Container
 - Port Mappings
 - Exec into Container
 - Simples Docker Image selbst bauen
- Empfehlenswerte Übersicht über nützliche Docker Befehle: Docker Cheatsheet
https://docs.docker.com/get-started/docker_cheatsheet.pdf

4. Weiterführendes Wissen: Docker Compose

- Startet **mehrere** Container **gleichzeitig**
- Verbindet diese automatisch über internes Netzwerk
→ **ermöglicht Kommunikation** zwischen Containern
- Ermöglicht **zentrales Starten** und **Stoppen** einer Anwendung
- Einsatzzweck: „**Orchestrierung**“ von Anwendungen, welche aus mehreren Containern bestehen
- Weiterführende Erklärung zu Docker Compose:
<https://docs.docker.com/compose/>

```
docker-compose.yml
  ▶ Run All Services
1  services:
  ▶ Run Service
2    backend:
3      build: ./backend
4      container_name: notes-backend
5      ports:
6        - "5000:5000"
7      volumes:
8        - notes_data:/data
9      restart: unless-stopped
10
  ▶ Run Service
11   frontend:
12     build: ./frontend
13     container_name: notes-frontend
14     ports:
15       - "8080:80"
16     depends_on:
17       - backend
18     restart: unless-stopped
19
20  volumes:
21    notes_data:
```

4. Weiterführendes Wissen: Docker Volumes

- Speichern Daten dauerhaft, auch wenn Container neu gebaut oder gelöscht werden
- Haupteinsatzzweck: Persistierung von Datenbanken
- Erzeugen des Volumes notes_data:

```
volumes:  
  notes_data:
```

- Ordner /data des Container Dateisystems wird im Volume notes_data persistiert:

```
volumes:  
  - notes_data:/data
```

4. Weiterführendes Wissen: Praxisteil 2

- Grundidee: Anwendung „Notizzettel“
 - Backend: Flask-App mit SQLite-Datenbank
 - Frontend: einfache HTML-Seite, lädt Daten des Backends in iFrame
 - docker-compose.yml: startet und verbindet die Container
 - Volume: speichert Notizen dauerhaft
- Wichtige Befehle:
 - Starten der Anwendung (=Bau der Images, Erzeugung und Start der Container)
 - ***docker compose up --build***
 - Stoppen der Container:
 - ***docker compose down***
 - Stoppen der Container inkl. Löschen des Volumes:
 - ***docker compose down -v***

4. Weiterführendes Wissen: Best Practices

- Dockerfiles minimal halten, kleine Images
- Multi-Stage Builds zur Reduzierung der Image-Größe
- Feste Versionierung der Base Images (tags)
- Keine sensiblen Daten/Passwörter im Image
- Modularität
- .dockerignore: verhindert, dass unnötige oder große Dateien ins Image kommen



UNIVERSITÄT
LEIPZIG

Fragen?

Tristan Scholz: scholz@studserv.uni-leipzig.de

Tobias Matzke: matzke@studserv.uni-leipzig.de